StrawSlot

Book the machine before the fire.

Backend Architecture & Operations Guide

Production-grade API · scaling design · security · deployment · measured performance

Team TrailBlazers

Adithya Phani Thota (Founder) · Sai Sivakesh Paladugu (CTO & Co-founder)

Cloud Counselage × DTI Social Impact Hackathon 2026 · Climate & Sustainability

Version 1.0 · 22 October 2026

1. What this is

This package is the complete server side of StrawSlot, rebuilt as a deployable service rather than a demo. It exposes the same scheduling engine the prototype uses, but behind a versioned REST API with real authentication, a relational database, a shared cache, a reliable SMS outbox, background workers, metrics and container/Kubernetes deployment files. The existing React front end works against it unchanged (the old `/api/...` paths are kept as an alias of `/api/v1/...`).

Property	Result
Language / runtime	TypeScript (strict ESM) on Node 22, Express 4
Datastore	PostgreSQL 16 (source of truth), Redis 7 (optional shared cache and rate limits)
API	36 operations, one declaration per route generates routing, validation, auth and the OpenAPI 3 spec
Tests	50 automated tests (11 engine, 39 integration/concurrency/outbox/cache/rate-limit) against a real Postgres and Redis – all passing
Processes	Stateless API replicas + separately scalable worker replicas, same Docker image
Deployment files	Dockerfile, docker-compose (2 API + worker + Postgres + Redis + nginx + Prometheus), Kubernetes manifests with autoscaling, GitHub Actions CI

Honesty note. Everything in this document describing behaviour was exercised by the test suite or the load tests in this sandbox. Performance numbers come from a 2-vCPU container with the load generator, API, Postgres and Redis all on the same machine, so they are conservative for a real deployment and are not a promise for any specific production hardware. Section 9 states the limits plainly, including the one that matters most (the scheduler's cost at very large district sizes).

2. Architecture at a glance

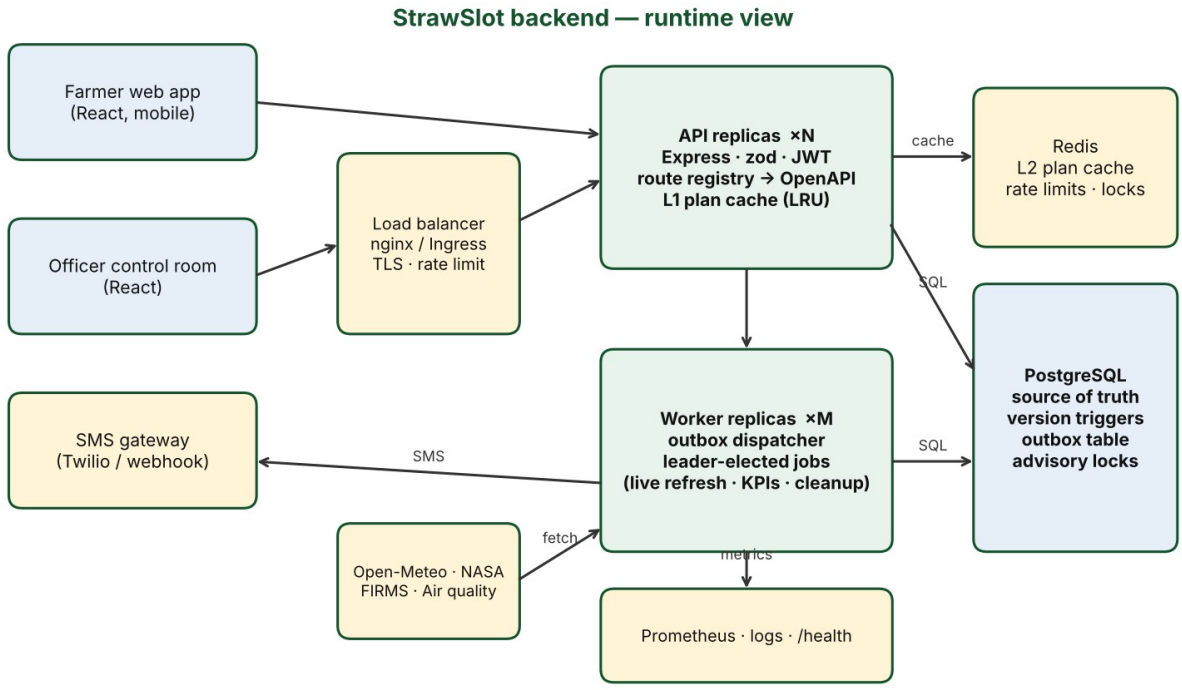


Figure 1 – Runtime view: stateless API replicas, workers, Postgres as the source of truth, Redis as an optional accelerator.

Design principles

- **Postgres is the only source of truth.** Every other layer (in-process cache, Redis, snapshots) can be lost at any time without losing data or correctness.
- **Compute once, share everywhere.** The district plan (who gets which machine on which day) is expensive to compute and cheap to read. It is computed once per data version and shared across requests, processes and servers.
- **Writes never wait on slow third parties.** SMS is written to an outbox in the same database transaction as the booking and delivered later by workers.
- **Every layer degrades instead of failing.** No Redis → in-process cache and in-memory rate limiting. SMS gateway down → retries with backoff. Live-data APIs down → clearly labelled fallback data.
- **Horizontal by default.** No instance holds state that another instance needs. Coordination uses database locks (advisory locks, SKIP LOCKED) rather than sticky sessions.

Code layout

```
src/
  config/       env.ts            zod-validated environment, production safety checks
  core/         errors, context (request-id), logger (pino), metrics (prom-client)
  db/           pool, forward-only migrations, bulk helpers, 001_init.sql
  domain/       engine.ts         the scheduler / risk / alerts / KPIs (pure, no I/O)
  repositories/ districts, snapshot, ops, users, tokens   (all SQL lives here)
  services/     planService (cache core), officerService, farmerService, auth,
                idempotency, liveService, liveFeeds, views, briefing
  jobs/         outbox (dispatcher), sms providers, scheduled jobs, runner
  http/         route registry, OpenAPI generator, middleware, routes/*
  server.ts     API process      worker.ts   worker process
tests/          engine, api, concurrency, outbox, ratelimit, redis, redis_down
deploy/         docker-compose, nginx, prometheus, k8s/*
loadtest/       autocannon runner, engine scaling benchmark
```

3. Data model

Every table carries `district_id`, and primary keys are composite with it. Adding a second district is a data operation, not a code change, and one district can never read another's rows.

Table	Purpose	Notes
districts	One row per district	`data_version` counter, `next_field_no` allocator
blocks, villages	Geography	Seeded for Sangrur
chcs, tractors, machines, vendors	Supply side: hiring centres, tractors, super-seeders/happy-seeders, straw buyers	Triggers bump `data_version`
fields	Registered fields (farmer, location, acres, harvest date, status)	`version` column for optimistic locking
pins	Officer overrides (field → machine from a date)	Triggers bump `data_version`
issues	Farmer-reported problems	Deliberately no trigger: reports do not change the plan
users, refresh_tokens	Accounts (scrypt hashes, lockout) and rotating refresh-token families	Roles: viewer < officer < admin
notifications	Transactional outbox for SMS	status, attempts, next_attempt_at, dedupe_key
idempotency_keys	Replay protection for farmer registration	Stores the first response
live_snapshots	Latest weather / air / fire data from the worker	Read by every API replica
kpi_history, audit_log, job_runs	Trends, who-did-what, background job history	Append-only

Why a version counter?

Row triggers on the plan-affecting tables increment `districts.data_version` inside the same transaction as the change. That single integer is the cache key for the whole plan. Any writer – this API, a script, a manual SQL fix – invalidates every replica's cache automatically, with no application code having to remember to do it. Issues are excluded on purpose so that a complaint does not throw away a perfectly good plan.

4. The plan cache – the heart of the scaling story

Computing a district plan is CPU-heavy; reading it is not. The design makes sure the computation happens as rarely as possible and never twice at the same time.

Layer	What it is	Behaviour
L1	In-process LRU (6 entries)	Microseconds; per replica
L2	Redis, gzip-compressed	Shared by all replicas; a freshly started replica warms from here instead of recomputing
Compute	The engine	Single-flight inside a process; a Redis `SET NX` lock makes other replicas wait for the result instead of duplicating work

Cache key: `plan:<district>:<data_version>:<liveStamp>:<today>`. The live stamp changes when new weather or fire data lands; the date changes at midnight IST, so deadlines never go stale.

- **Consistent reads.** The version is read inside one repeatable-read snapshot together with the state, so a plan is never labelled with a version it was not computed from.

- **Group commit for writes.** A write needs a fresh plan in its response. A burst of writes shares one in-flight computation plus at most one follow-up. In the test, 40 farmers booking at the same instant caused 8 computations, not 40, and every response contained that farmer's own correct slot.
- **Cheap version lookup.** `VERSION_CACHE_MS` (default 250 ms) bounds how often readers ask Postgres for the version.
- **HTTP-level caching.** Plan-derived routes return a plan-aware `ETag`; clients that revalidate get `304 Not Modified` without a body.
- **Per-plan memoised views.** The 139 KB officer overview and the field list are built once per plan object instead of once per request (measured gain: 1.6–3.5×).

5. Reliability mechanisms

5.1 Transactional outbox for SMS

When a farmer books or a plan changes for them, the SMS is inserted into `notifications` in the same transaction as the change. Either both happen or neither does – there is no "booked but never told" and no "told about a booking that rolled back".

- Workers claim rows with `FOR UPDATE SKIP LOCKED`: any number of workers run in parallel and no message is claimed twice (tested: 6 dispatchers, 60 messages, each sent exactly once).
- The text is rendered at send time from the latest plan, in the farmer's language (English / Hindi / Punjabi), so a delayed message never carries stale dates.
- Failures retry with exponential backoff (30 s doubling, capped at 1 h) and become `dead` after 6 attempts, visible to officers in the Notifications view.
- A row stuck in `sending` for more than 5 minutes (worker crashed mid-send) is recovered automatically.
- A `dedupe_key` stops duplicate enqueues. Providers: `console` (dev), signed `webhook` (HMAC), `twilio`.

5.2 Idempotent registration

Mobile networks drop responses. The farmer app can send an `Idempotency-Key` header; a retry with the same key returns the original result instead of creating a second field. The key is claimed inside the same transaction as the insert, so two simultaneous retries cannot both win.

5.3 Concurrency control

- **Optimistic locking** on fields: an officer edit carries `expectedVersion`; if someone else changed the field first the API answers `409`. Test: 6 simultaneous edits of the same version → exactly 1 succeeds, 5 get 409.
- **Per-district advisory lock** around officer read-modify-write operations (auto-fix, reassign), so two officers cannot interleave and overwrite each other's pins.
- **Unique, gap-tolerant field ids** from an atomic allocator (40 simultaneous bookings → 40 unique ids).

5.4 Background jobs without a scheduler service

Every worker runs the outbox loop. The periodic jobs – live-data refresh (weather, air quality, fire hotspots), KPI snapshot, cleanup, demo reseed – run only on the instance that holds the Postgres advisory lock 727002 (taken on a dedicated connection). If that worker dies the lock is released and another takes over within one interval. Each run is recorded in `job_runs`.

5.5 Graceful behaviour at the edges

- **Shutdown:** on SIGTERM readiness turns red first, the process waits a grace delay so the load balancer stops sending traffic, then drains in-flight requests and closes pools.

- **Readiness vs liveness:** `/health/live` never touches dependencies; `/health/ready` checks database, migration state, pool and Redis. Redis being down reports `degraded` and stays ready – a cache outage must not take the API out of rotation (tested).
- **Timeouts:** per-request timeout (25 s), database statement timeout, keep-alive tuned above typical load-balancer idle time.

6. Security

Area	Implementation
Passwords	scrypt with per-user salt; constant-time comparison; no plaintext anywhere
Login abuse	5 failures lock the account for 15 minutes (HTTP 423); per-IP login rate limit (20 per 15 min)
Tokens	Short-lived JWT access token (HS256, `jose`); rotating refresh tokens stored hashed. Re-using an old refresh token is treated as theft and revokes the whole token family (tested)
Authorisation	Roles viewer < officer < admin enforced per route from the same declaration that defines the route; every officer action writes the audit log
Input	Every body, query and path parameter validated by zod; unknown fields stripped; 1 MB body limit; consistent error shape with request id
Transport / headers	helmet, CORS allow-list, trust-proxy aware client IPs, `Retry-After` and standard `RateLimit-*` headers
Rate limits	Global 600/min, login 20/15 min, farmer writes 40/hour per client. Fixed-window counters in Redis (shared across replicas) with in-memory fallback; fails open if Redis is unreachable
Secrets	Production start-up refuses weak config: JWT secret ≥ 32 chars, bootstrap passwords required, SMS provider settings complete
Logging	Structured JSON (pino) with credential redaction; request id on every line and every error response
Metrics	`/metrics` protected by bearer token when `METRICS_TOKEN` is set; blocked at the nginx edge
Container	Multi-stage image, non-root user, production dependencies only

Known limitations to address before a government rollout: JWT secret rotation is manual; there is no MFA for officer accounts; access tokens are 12 h by default only because the demo front end has no refresh flow (set `ACCESS_TOKEN_TTL_MIN=15` in production and add a refresh call in the UI); farmer registration is phone-number based without OTP verification (an OTP step through the SMS gateway is the natural next feature).

7. API reference

Canonical base path `/api/v1` (also served at `/api`). Interactive documentation: `/docs` (Swagger UI) and machine-readable `/openapi.json`. Errors always look like `{ "error": "message", "code": "machine_code", "requestId": "…" }`.

Method	Path	Purpose	Access
GET	`/health`	Liveness with a few facts	public
GET	`/meta`	District, blocks, villages, hiring centres, machine types and scheduling assumptions	public
GET	`/public/stats`	Headline numbers for the landing page	public
GET	`/live/weather`	Latest stored weather snapshot	public
GET	`/live/air`	Latest stored air-quality snapshot	public
GET	`/live/fires`	Latest stored fire-hotspot snapshot (add ?scope=district for district points only)	public
POST	`/farmer/fields`	Register a field and get the booked slot back. Send an Idempotency-Key header to make retries safe.	public
GET	`/farmer/lookup`	All slots registered under a mobile number	public
POST	`/farmer/issues`	Report a problem with a booking	public
POST	`/auth/login`	Sign in. Returns a short-lived access token and a rotating refresh token.	public
POST	`/auth/refresh`	Exchange a refresh token for a new access token and a new refresh token (the old one stops working)	public
POST	`/auth/logout`	Revoke a refresh token family	public

GET	/auth/me	Who am I	any signed-in
GET	/admin/overview	Control-room overview: KPIs, map data, weather, air, fires	officer+
GET	/admin/fields	Fields with plan and risk (search, filter, optional pagination)	officer+
GET	/admin/fields/{id}	One field: plan, SMS text, reports, version	officer+
POST	/admin/fields/{id}/status	Mark a field cleared, burned or pending (optional expectedVersion for optimistic locking)	officer+
GET	/admin/dispatch	Machine-by-day dispatch board for four weeks	officer+
GET	/admin/alerts	At-risk fields with ranked fixes, and farmer reports	officer+
POST	/admin/reassign	Pin a field to a machine from a date (officer override)	officer+
POST	/admin/autofix	Apply the first suggested fix that helps the field without hurting the district	officer+
POST	/admin/autofix-all	Sweep the most urgent alerts and apply every net-positive fix	officer+
DELETE	/admin/pins/{fieldId}	Remove an officer override	officer+
POST	/admin/issues/{id}/resolve	Resolve a farmer report with a note	officer+
POST	/admin/machines/{id}	Put a machine into or out of maintenance	officer+
GET	/admin/registry	Hiring centres, machines with utilisation, straw buyers	officer+
POST	/admin/whatif	Simulate adding machines or tractors. Nothing is saved.	officer+
GET	/admin/briefing	Today's briefing	officer+
GET	/admin/log	Audit trail, newest first (cursor: ?before=<id>)	officer+
GET	/admin/trends	Daily KPI history	officer+
GET	/admin/notifications	Outbox: queued, sent and failed messages to farmers	officer+
GET	/admin/export/schedule.csv	Download the schedule as CSV	officer+
POST	/admin/reset	Rebuild the simulated demo season (demo district only)	officer+
GET	/admin/users	List users in the district	admin
POST	/admin/users	Create a user	admin
PATCH	/admin/users/{id}	Change role, display name, password or disable a user	admin

Operational endpoints (outside `/api`): `/health/live`, `/health/ready`, `/metrics`, `/openapi.json`, `/docs`.

Quick start for a reviewer

```
# 1. sign in (demo credentials: officer / sangrur2026)
curl -s -XPOST localhost:8080/api/v1/auth/login -H 'content-type: application/json' \
     -d '{"username":"officer","password":"sangrur2026"}'
# 2. use the accessToken
curl -s localhost:8080/api/v1/admin/overview -H "authorization: Bearer $TOKEN"
# 3. a farmer books a slot (no login), safe to retry
curl -s -XPOST localhost:8080/api/v1/farmer/fields -H 'content-type: application/json' \
     -H 'Idempotency-Key: abc-123' -d
'{"farmerName":"Test","phone":"9876543210","villageId":"…","acres":4,...}'
```

8. Observability

Signal	Where	Examples
Logs	stdout, JSON	request id, route, status, duration, user; secrets redacted
Metrics	`/metrics` (API) and worker port 9100	http request duration histogram by route/status; `schedule_compute_seconds`; `plan_cache_events_total{layer,result}`; job run counters; notifications sent/failed; outbox depth gauge; rate-limited and login-attempt counters
Health	`/health/live`, `/health/ready`	used by Kubernetes probes and the load balancer
Audit	`audit_log` table, `/admin/log`	every officer action with actor and time
Jobs	`job_runs` table	start, finish, outcome of every scheduled run

Suggested alerts

- p95 request latency > 500 ms for 5 minutes
- `plan_cache_events_total{result="miss"}` rate rising while traffic is flat (cache not holding)
- Outbox depth growing for 10 minutes, or any `dead` notification
- No successful `live-refresh` job for 3 hours
- Readiness failures on more than one replica

9. Performance and scaling – measured, with limits

9.1 Load test (sandbox, 2 vCPU, everything on one box)

Tool: autocannon, 64 connections, 10 seconds per scenario, rate limiting disabled for the test, single API process, Redis enabled. The load generator competes with the server for the same two CPUs, so a dedicated load generator and a real server would show higher numbers.

Scenario	Requests/s	p50 ms	p97.5 ms	p99 ms	Errors
Health (liveness)	5,555	10	22	25	0
Public stats (anonymous, cached plan)	2,564	19	68	96	0
Officer overview (139 KB payload)	673	93	143	154	0
Officer field list (50 rows)	901	60	184	234	0
Mixed officer + public reads	1,133	42	84	90	0

The officer overview improved from 415 to 673 requests/s after memoising the view per plan. The remaining cost is serialising and compressing a 139 KB document; paging the map layer or sending a delta would raise it further.

9.2 What scales out, and how

Pressure	Lever	Why it works
More read traffic	Add API replicas (HPA on CPU)	Stateless; plans come from Redis; Postgres sees only a cheap version lookup
More SMS volume	Add worker replicas	SKIP LOCKED gives disjoint batches
More writes	Postgres vertical scale, then read replicas for GET-only paths	Writes are small, transactional and indexed by district
More districts	Add rows; shard by district later if needed	Every table is keyed by district

9.3 The honest limit: scheduler cost grows faster than linearly

The scheduling engine is a least-slack-first simulation. We measured one full plan computation as the district grows (machines scaled in proportion, so each machine still serves the same number of fields):

Fields in one district	Machines	One full computation
640 (the Sangrur seed)	23	≈ 50 ms warm (≈ 170 ms first call)
1,280	46	≈ 0.2 s
2,560	92	≈ 1.4 s
5,120	184	≈ 5.3 s

Because the plan is cached by data version, this cost is paid once per change, not once per request – but past a few thousand fields a busy district would recompute noticeably often, and the computation blocks one CPU core. **This is the single most important scaling limit, and we are stating it rather than hiding it.** The path forward is already clear because districts decompose naturally:

- **Shard the plan per block** (Sangrur has several): each block's fields mostly compete for the same nearby hiring centres, so block-level plans are small, can be computed independently and cached separately, turning the super-linear curve into a sum of small ones.
- **Incremental recomputation:** only re-plan fields whose machine pool changed.
- **Worker-thread pool** so a long computation never blocks the event loop of a request-serving process.
- Because the engine is a pure function over a state object, none of these require changing the API or the database.

Related measurement: the "auto-fix all" action is CPU-bound (about 4–5 s per call on this sandbox, time-boxed to 6.5 s inside the request). Concurrent calls serialise correctly behind the district lock; three simultaneous calls finished cleanly in about 14 s. For production it should become an asynchronous job with a status endpoint.

10. Deployment

10.1 Single host: Docker Compose

```
cd deploy && cp ../.env.example .env      # edit secrets
docker compose up --build                  # api x2, worker, postgres, redis, nginx, prometheus
curl localhost:8080/health/ready
```

A one-shot `migrate` service applies migrations before the API and worker start. nginx round-robins the API replicas, applies per-IP request limiting and hides `/metrics`.

10.2 Kubernetes

Manifest	What it gives you
`config.yaml`	ConfigMap and (placeholder) Secret – replace with your secret manager
`api.yaml`	Deployment (3 replicas, zero-downtime rolling update), startup/liveness/readiness probes, resource limits, preStop delay, Service, HorizontalPodAutoscaler (3–12 replicas at 65% CPU), PodDisruptionBudget (min 2 available)
`worker.yaml`	Deployment of 2 workers with liveness probe and metrics port
`migrate-job.yaml`	Run before every rollout; advisory-locked and idempotent
`ingress.yaml`	TLS ingress with cert-manager annotation

10.3 Configuration reference

Variable	Purpose / default
DATABASE_URL, REDIS_URL	Connections (Redis optional)
JWT_SECRET	≥ 32 chars in production
ACCESS_TOKEN_TTL_MIN, REFRESH_TOKEN_TTL_DAYS	720 / see env.ts (use 15 in production)
RUN_WORKER_IN_API	true for single-process setups; false when workers run separately
MIGRATE_ON_BOOT, BOOTSTRAP_SEED	Auto-migrate and create demo district/users on first boot
VERSION_CACHE_MS, PLAN_CACHE_TTL_S	Cache tuning (250 ms / TTL)
RATE_LIMIT_ENABLED, RATE_LIMIT_*	Limits listed in section 6
CORS_ORIGINS, TRUST_PROXY, METRICS_TOKEN	Edge configuration
FIRMS_MAP_KEY, ANTHROPIC_API_KEY	Optional live fire data and AI briefing
SMS_PROVIDER, SMS_WEBHOOK_URL/SECRET, TWILIO_*	Message delivery

10.4 Runbook

Situation	Action
Roll out a new version	Run migrate job → rolling update. Migrations are forward-only; ship additive changes first
Redis outage	No action needed; API reports `degraded`, falls back to in-process cache. Expect more plan computations until it returns
SMS backlog	Check `/admin/notifications` and the outbox gauge; scale workers; check gateway status. Dead rows can be re-queued after the cause is fixed
Stale weather/fire data	Check `job_runs` for `live-refresh`; confirm outbound network and FIRMS key
Slow officer screens	Check `schedule_compute_seconds` and cache miss rate; see section 9.3

Suspected token theft	Disable the user (PATCH /admin/users/{id}); refresh families revoke automatically on reuse

11. Testing

Suite	What it proves
engine (11)	The ported scheduler, risk score, alerts and KPIs behave exactly as in the prototype
api (23)	Health, auth, refresh rotation and reuse detection, RBAC, lockout, validation, the full farmer flow, idempotency, ETag/cache correctness, every officer action, what-if, CSV, OpenAPI, metrics
concurrency (4)	40 simultaneous bookings → unique ids and 8 computations; parallel auto-fix serialises; optimistic locking; 200 cached reads → 0 computations
outbox (6)	Same-commit enqueue, rendered-at-send text, dedupe, retry/backoff/dead-letter, exclusive claims, crash recovery
ratelimit (2), redis (3), redis_down (1)	Limits and headers; L2 cache and limits shared via Redis; full function with Redis unreachable

Run everything with `npm test` (needs a local Postgres test database; Redis for the Redis suite). CI runs typecheck, tests and the build on every push.

12. Why this matters for the market

- **Credibility with a district administration:** an auditable system with roles, an action log, and a clear failure story is what a government buyer's IT team asks about before the first pilot.
- **Cost to serve is low.** A single small VM comfortably handles a district's officers and farmers; the cache design means traffic growth costs read replicas, not compute.
- **Multi-district ready.** Tenancy is in the schema. The path from one district to Punjab is configuration, plus the block-sharding work described in 9.3.
- **Honest roadmap:** OTP verification, officer MFA, asynchronous bulk actions and block-sharded planning are the four items between this pilot-grade backend and statewide scale.

13. Quick start

```
npm install
# local Postgres + Redis, then:
./dev-up.sh                    # API + in-process worker on :8080
npm test                       # 50 tests
npm run loadtest               # autocannon scenarios
npx tsx loadtest/engine-scale.ts
npm run build && node dist/server.js
```

Demo logins (development only): officer / sangrur2026, admin / admin-sangrur-2026. In production these come from environment variables and the process refuses to start without them.